

INFORME INTEGRAL

UML aplicado a POO: clases, atributos, métodos y visibilidad

Proyecto: Sistema de Gestión para una Tienda de Ropa
Unidad didáctica: Programación Orientada a Objetos (POO)
Estudiante: SONIA JHAKELIN OCAS AZAÑERO
Docente: Ing. Carlos Abilio Angulo Zegarra

Paiján – La Libertad, Perú | 2026

1. Introducción

Este informe integra el trabajo previamente validado de identificación del modelado del dominio y especificación de requerimientos con la actividad de construcción del diagrama de clases UML. Se conserva el contenido anterior —problema, participantes, procesos, entidades, requerimientos, historias de usuario y diagramas— y se amplía el modelo de clases con tipos de datos, visibilidades, multiplicidades, comparación conceptual, decisiones técnicas y bitácora de IA.

1.1 Objetivo general

Identificar y modelar las entidades principales del sistema, especificar sus requerimientos y representar su estructura y comportamiento mediante diagramas UML.

1.2 Objetivos específicos

- Analizar el problema, participantes y procesos de la tienda.
- Identificar entidades, atributos, responsabilidades y relaciones.
- Especificar requerimientos funcionales y no funcionales con prioridad y trazabilidad.
- Redactar historias de usuario con criterios verificables.
- Representar dominio, casos de uso, clases y secuencia de venta.
- Precisar tipos, visibilidades, operaciones, multiplicidades y justificar decisiones.

2. Contexto y comprensión del problema

2.1 Problema central

Deficiente control de productos, clientes, ventas y stock debido al uso de registros manuales y a la falta de un sistema centralizado. Esto puede ocasionar errores en registros y cantidades disponibles, diferencias entre stock registrado y físico, y dificultades para consultar y seguir las operaciones.

2.2 Participantes principales

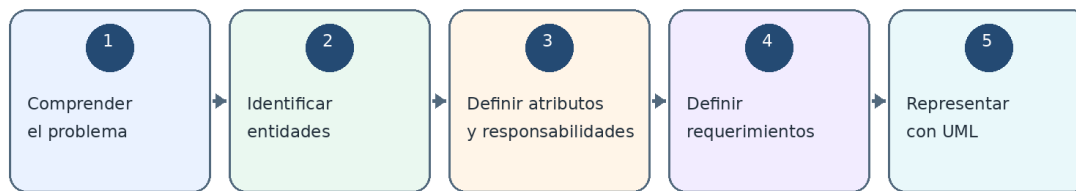
Participante	Participación
Administrador	Administra sistema, productos y usuarios; consulta ventas.
Vendedor	Consulta productos y stock, registra clientes y ventas.
Cliente	Realiza la compra de productos.

2.3 Procesos identificados

N.º	Proceso
1	Registrar productos
2	Consultar productos
3	Controlar y actualizar stock
4	Registrar clientes
5	Registrar ventas
6	Verificar disponibilidad
7	Calcular total de venta
8	Actualizar stock automáticamente
9	Generar comprobante
10	Consultar ventas realizadas

Figura 1. Fases del análisis orientado a objetos.

PROCESO DE ANÁLISIS ORIENTADO A OBJETOS



Base: fases de la guía práctica proporcionada.

3. Modelado inicial del dominio

Se identifican los objetos principales del problema y se determina qué información y responsabilidades corresponden a cada uno. Los sustantivos que funcionan como datos se modelan como atributos, no como entidades independientes.

3.1 Identificación de entidades

Elemento	Decisión	Justificación
Usuario	Entidad	Accede y registra operaciones.
Producto	Entidad	Concentra características y existencias.
Cliente	Entidad	Se relaciona con una compra.
Venta	Entidad	Representa la operación comercial.
Detalle de Venta	Entidad	Representa productos, cantidades, precios y subtotales.
Comprobante	Entidad	Representa el comprobante de una venta.
Stock	Atributo de Producto	Dato del producto.
Categoría, talla, color, precio	Atributos de Producto	Características/datos del producto.
Cantidad	Atributo de Detalle de Venta	Dato por producto vendido.

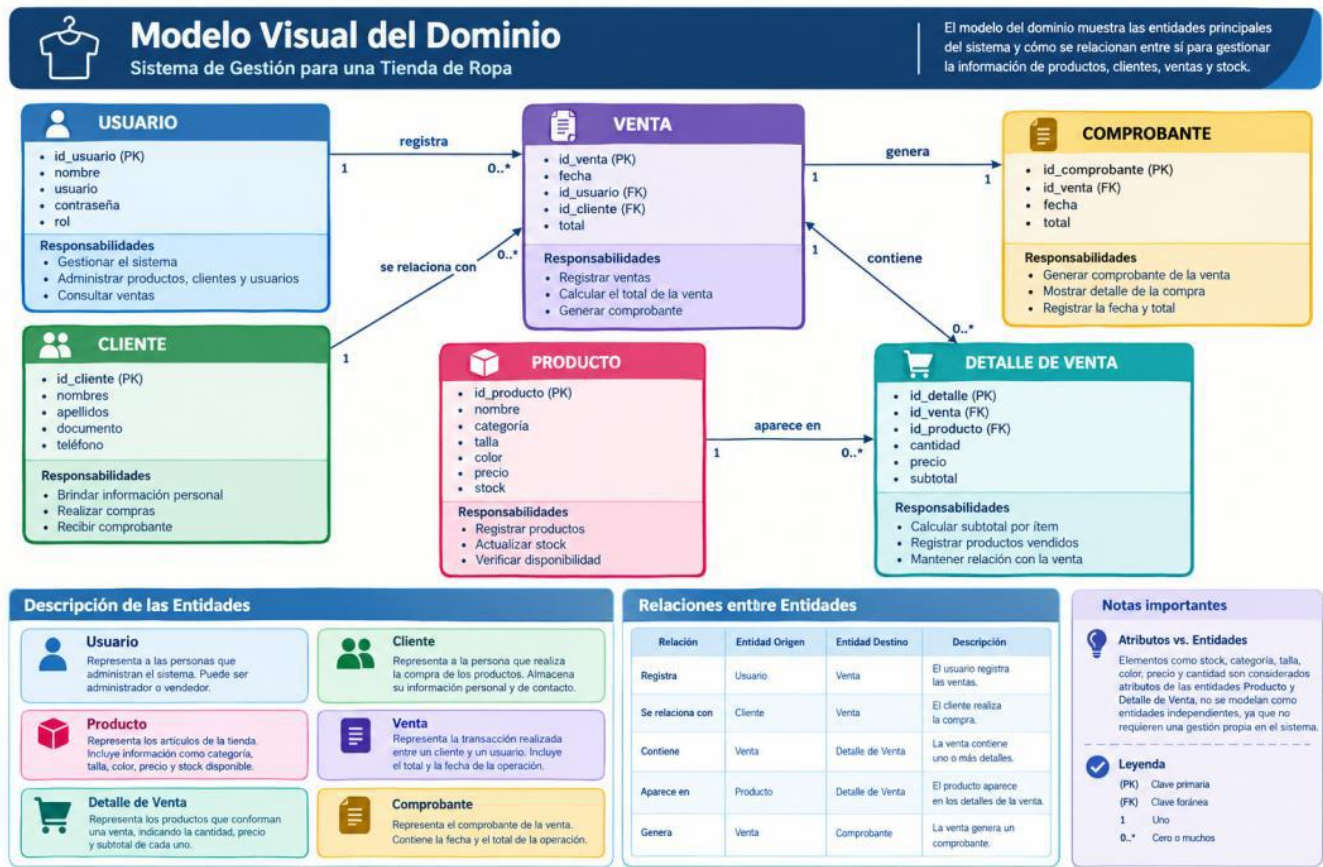
3.2 Catálogo de entidades, atributos y responsabilidades

Entidad	Atributos principales	Responsabilidades	Relaciones
Usuario	id_usuario, nombre, usuario, contraseña, rol	Iniciar sesión, controlar acceso, registrar ventas.	Venta
Producto	id_producto, nombre, categoría, talla, color, precio, stock	Registrar, modificar, consultar y controlar existencias.	Detalle de Venta
Cliente	id_cliente, nombres, apellidos, documento, teléfono	Registrar y consultar clientes.	Venta
Venta	id_venta, fecha, id_usuario, id_cliente, total	Registrar ventas, calcular total, conservar historial.	Usuario, Cliente, Detalle, Comprobante
Detalle de Venta	id_detalle, id_venta, id_producto, cantidad, precio, subtotal	Registrar productos, cantidades, precios y subtotales.	Venta, Producto
Comprobante	id_comprobante, id_venta, fecha, total	Generar y conservar comprobante.	Venta

3.3 Relaciones del dominio

Relación	Representación
Usuario—Venta	Usuario registra ventas.
Cliente—Venta	Cliente se relaciona con la venta.
Venta—Detalle de Venta	Venta contiene sus detalles.
Producto—Detalle de Venta	Producto aparece en los detalles.
Venta—Comprobante	Comprobante corresponde a una venta.

3.4 Modelo visual del dominio



4.2 Requerimientos no funcionales

ID	Descripción	Prioridad	Entidades relacionadas
RNF-01	Fácil de utilizar.	Alta	Usuario
RNF-02	Proteger información almacenada.	Alta	Usuario, Producto, Cliente, Venta
RNF-03	Responder rápidamente a consultas.	Media	Producto, Cliente, Venta
RNF-04	Mantener integridad de datos.	Alta	Producto, Cliente, Venta, Detalle
RNF-05	Permitir acceso solo a usuarios autorizados.	Alta	Usuario
RNF-06	Contar con autenticación.	Alta	Usuario
RNF-07	Permitir copias de seguridad.	Media	Base de datos

4.3 Relación entre problema, procesos y requerimientos

Necesidad	Proceso relacionado	Requerimientos
Controlar productos/stock	Registrar/consultar productos; actualizar stock	RF-02, RF-05, RF-07
Evitar errores en ventas	Registrar venta, verificar disponibilidad, calcular total	RF-04, RF-05, RF-06
Gestionar clientes	Registrar clientes	RF-03
Conservar constancia	Generar comprobante	RF-08
Dar seguimiento	Consultar ventas	RF-09
Controlar acceso	Iniciar sesión y administrar usuarios	RF-01, RF-10, RNF-05, RNF-06

5. Historias de usuario

ID	Historia	Criterios de aceptación
HU-01	Como vendedor, quiero registrar una venta para conservar la operación.	Identificar cliente; agregar productos/cantidades; verificar stock; calcular total; registrar venta.
HU-02	Como administrador, quiero gestionar productos para mantener actualizada la tienda.	Registrar, modificar y consultar producto; mantener stock.
HU-03	Como vendedor, quiero consultar disponibilidad para evitar ventas sin existencias.	Consultar producto; mostrar stock; revisar antes de confirmar.
HU-04	Como vendedor, quiero generar comprobante para conservar constancia.	Venta registrada; comprobante asociado; fecha y total incluidos.

5.1 Explicación de las historias

Las historias cubren gestión de productos, disponibilidad, registro de ventas y comprobantes; sus necesidades corresponden a los requerimientos funcionales.

6. Diagramas UML del informe anterior

6.1 Diagrama de casos de uso

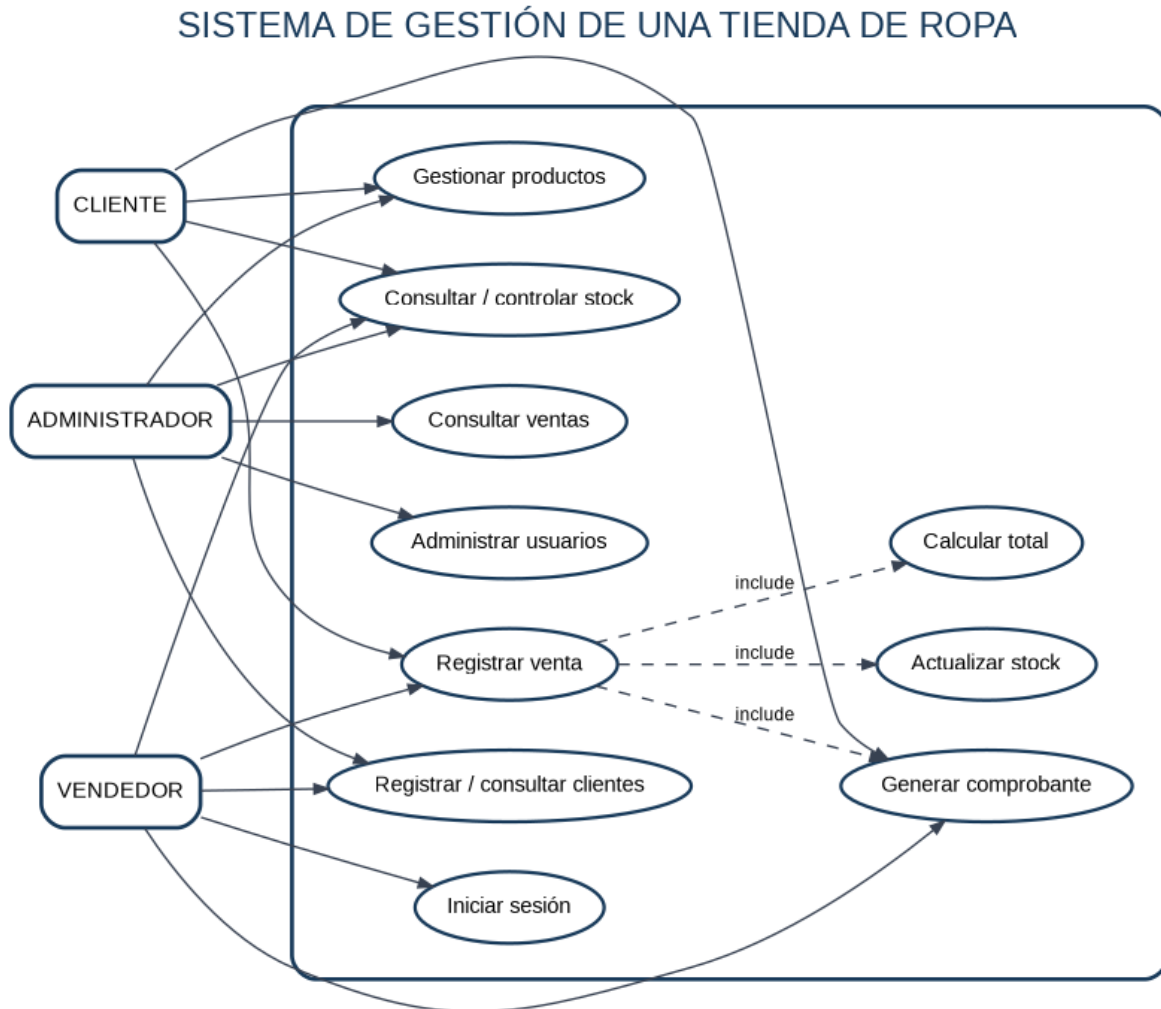


Figura 3. Casos de uso: actores y funcionalidades principales.

El Administrador gestiona productos, usuarios, clientes y stock, y consulta ventas. El Vendedor consulta productos/stock, registra clientes, ventas y comprobantes. El Cliente participa en la compra. Registrar venta se relaciona con cálculo del total, actualización del stock y generación del comprobante.

6.2 Diagrama de clases previo

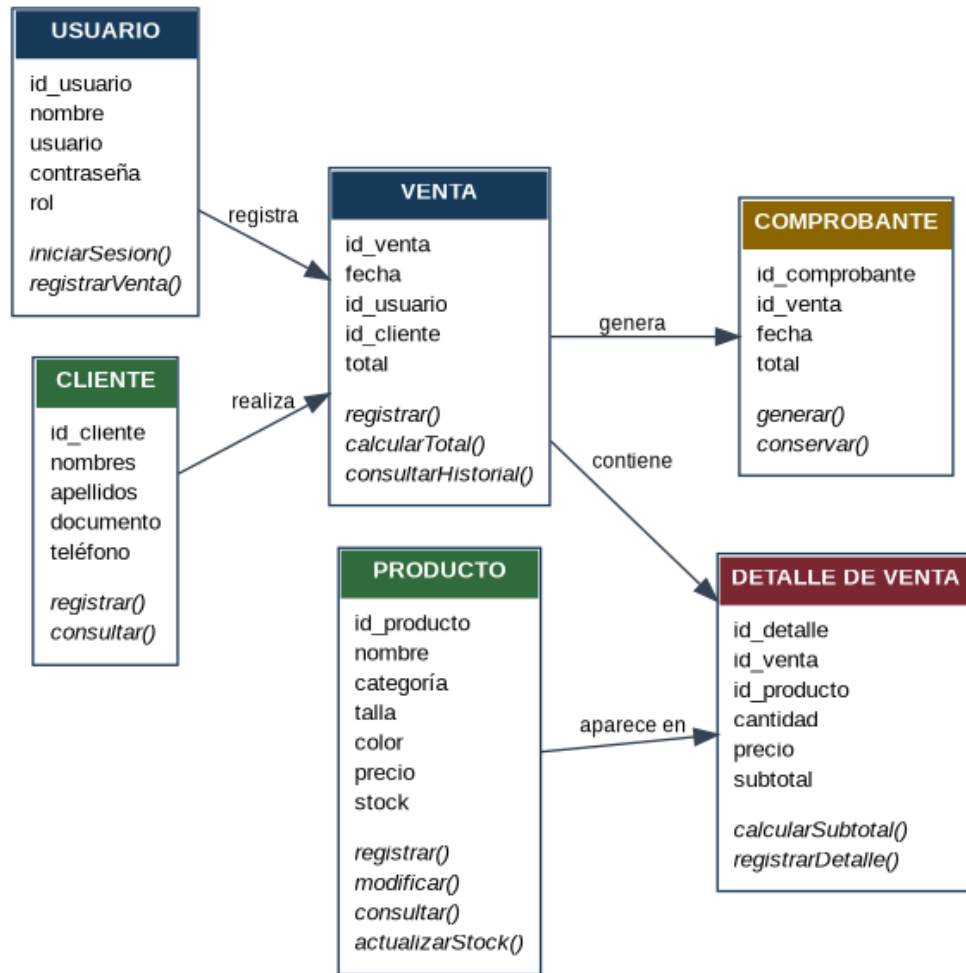


Figura 4. Diagrama de clases incluido en el informe previo.

Se conserva como evidencia del trabajo anterior. La versión ampliada de la sección 7 incorpora tipos, visibilidades y multiplicidades.

6.3 Diagrama de secuencia

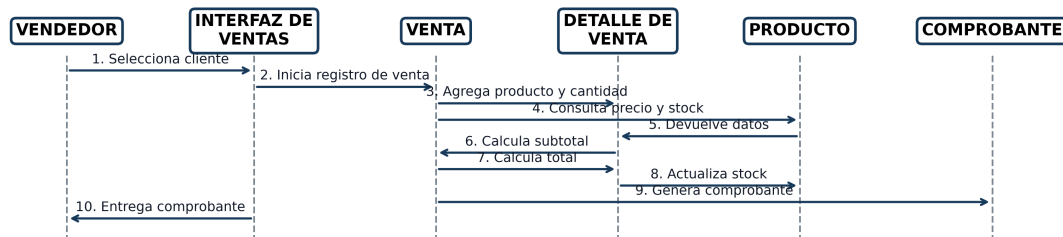


Figura 5. Secuencia UML del registro de una venta.

El flujo selecciona cliente, inicia venta, agrega productos y cantidades, consulta precio y stock, calcula subtotal/total, actualiza stock y genera comprobante.

7. Actividad ampliada: diagrama de clases UML

Se retoma el modelo del dominio y los requerimientos validados. Se identifican atributos con tipo, operaciones, visibilidad y multiplicidades.

7.1 Notación UML aplicada

Notación	Significado
+	Público; usado en operaciones.
-	Privado; usado en atributos.
#	Protegido; no se utiliza al no definirse herencia.
1	Exactamente una instancia.
0..*	Cero o muchas instancias.
1..*	Una o muchas instancias.
0..1	Cero o una instancia.
◆	Composición Venta–Detalle de Venta.

7.2 Atributos, tipos y visibilidad

Usuario

Atributo	Tipo	Visibilidad
id_usuario	Integer	- privada
nombre	String	- privada
usuario	String	- privada
contraseña	String	- privada
rol	String	- privada

Producto

Atributo	Tipo	Visibilidad
id_producto	Integer	- privada
nombre	String	- privada
categoría	String	- privada
talla	String	- privada
color	String	- privada
precio	Decimal	- privada
stock	Integer	- privada

Cliente

Atributo	Tipo	Visibilidad
id_cliente	Integer	- privada
nombres	String	- privada
apellidos	String	- privada
documento	String	- privada
teléfono	String	- privada

Venta

Atributo	Tipo	Visibilidad
id_venta	Integer	- privada
fecha	Date	- privada
id_usuario	Integer	- privada
id_cliente	Integer	- privada
total	Decimal	- privada

Detalle de Venta

Atributo	Tipo	Visibilidad
id_detalle	Integer	- privada
id_venta	Integer	- privada
id_producto	Integer	- privada
cantidad	Integer	- privada
precio	Decimal	- privada
subtotal	Decimal	- privada

Comprobante

Atributo	Tipo	Visibilidad
id_comprobante	Integer	- privada

id_venta	Integer	- privada
fecha	Date	- privada
total	Decimal	- privada

7.3 Operaciones principales y visibilidad

Usuario

Operación	Visibilidad
+ iniciarSesion()	Pública
+ controlarAcceso()	Pública

Producto

Operación	Visibilidad
+ registrarProducto()	Pública
+ modificarProducto()	Pública
+ consultarProducto()	Pública
+ controlarStock()	Pública

Cliente

Operación	Visibilidad
+ registrarCliente()	Pública
+ consultarCliente()	Pública

Venta

Operación	Visibilidad
+ registrarVenta()	Pública
+ calcularTotal()	Pública
+ consultarVentas()	Pública

Detalle de Venta

Operación	Visibilidad
+ registrarDetalle()	Pública
+ calcularSubtotal()	Pública

Comprobante

Operación	Visibilidad
+ generarComprobante()	Pública
+ conservarComprobante()	Pública

Las operaciones se asignan a la clase responsable: Usuario gestiona autenticación y acceso; Venta registra y consulta las ventas. No se añaden parámetros ni tipos de retorno no definidos en los requerimientos.

7.4 Asociaciones y multiplicidades

Asociación	Multiplicidad	Interpretación
Usuario—Venta	Usuario 1 / Venta 0..*	Cada venta la registra un usuario; un usuario registra cero o muchas ventas.
Cliente—Venta	Cliente 1 / Venta 0..*	Cada venta corresponde a un cliente; un cliente puede tener cero o muchas ventas.
Venta—Detalle	Venta 1 / Detalle 1..*	Una venta contiene uno o más detalles; composición.
Producto—Detalle	Producto 1 / Detalle 0..*	Cada detalle refiere un producto; un producto puede aparecer en varios detalles.
Venta—Comprobante	Venta 1 / Comprobante 0..1	Una venta puede tener cero o un comprobante.

7.5 Diagrama UML de clases ampliado y corregido

DIAGRAMA DE CLASES UML — SISTEMA DE GESTIÓN PARA TIENDA DE ROPA

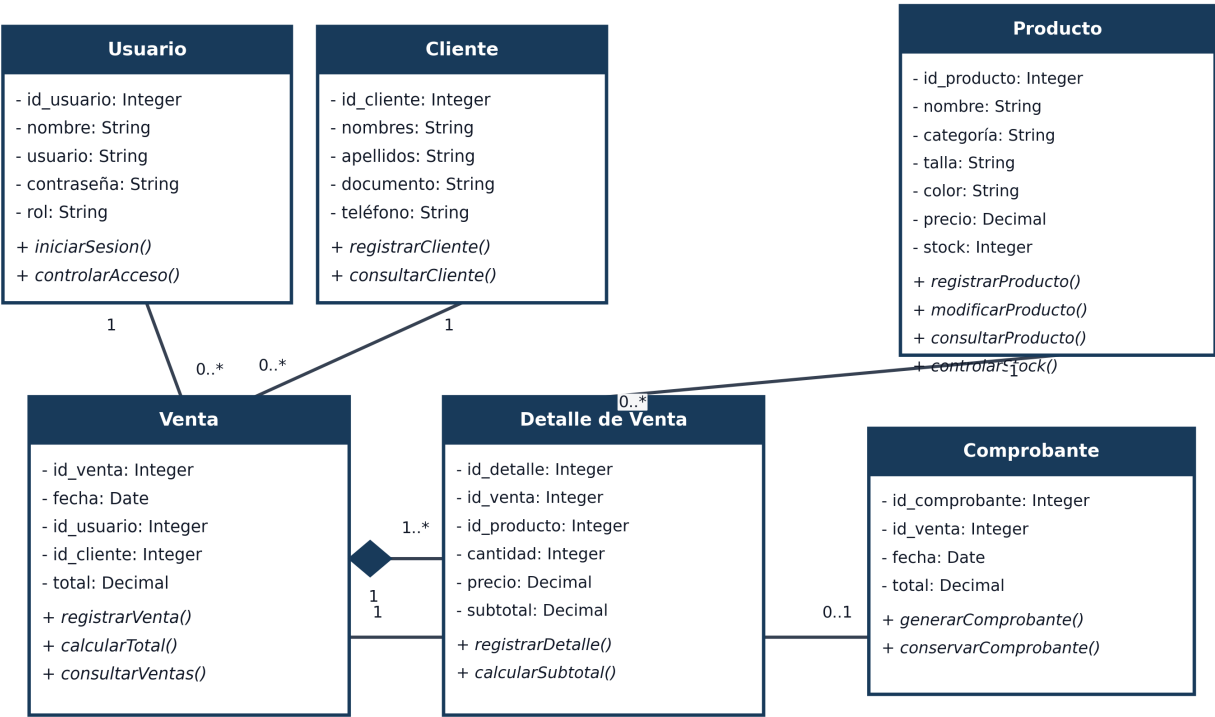


Figura 6. Diagrama de clases ampliado: seis clases, atributos, operaciones, visibilidades y multiplicidades.

Las asociaciones muestran sus multiplicidades en ambos extremos. La composición indica que cada Venta contiene uno o más Detalles de Venta. Se evita duplicar registrarVenta() en Usuario, pues esa responsabilidad corresponde a Venta.

Tabla complementaria: clases, atributos, métodos y visibilidad

Clase	Atributos (tipo de dato)	Visibilidad	Métodos / Operaciones	Visibilidad
Usuario	- id_usuario: Integer - nombre: String - usuario: String - contraseña: String - rol: String	- Privada	+ iniciarSesion() + controlarAcceso()	+ Pública
Cliente	- id_cliente: Integer - nombres: String - apellidos: String - documento: String - teléfono: String	- Privada	+ registrarCliente() + consultarCliente()	+ Pública
Producto	- id_producto: Integer - nombre: String - categoría: String - talla: String - color: String - precio: Decimal - stock: Integer	- Privada	+ registrarProducto() + modificarProducto() + consultarProducto() + controlarStock()	+ Pública
Venta	- id_venta: Integer - fecha: Date - id_usuario: Integer - id_cliente: Integer - total: Decimal	- Privada	+ registrarVenta() + calcularTotal() + consultarVentas()	+ Pública
Detalle de Venta	- id_detalle: Integer - id_venta: Integer	- Privada	+ registrarDetalle() + calcularSubtotal()	+ Pública

	- id_producto: Integer - cantidad: Integer - precio: Decimal - subtotal: Decimal			
Comprobante	- id_comprobante: Integer - id_venta: Integer - fecha: Date - total: Decimal	- Privada	+ generarComprobante() + conservarComprobante()	+ Pública

7.6 Comparación con el modelo conceptual

Modelo conceptual	Representación en clases	Resultado
Usuario	Datos de acceso y operaciones de autenticación/registro.	Se conserva.
Producto	Identificación, características, precio y stock.	Se conserva.
Cliente	Identificación y contacto.	Se conserva.
Venta	Fecha, total y asociaciones con usuario/cliente.	Se conserva y detalla.
Detalle de Venta	Producto, cantidad, precio y subtotal.	Se conserva y conecta Venta con Producto.
Comprobante	Identificador, fecha y total asociado a Venta.	Se conserva.

El diagrama conserva las seis entidades y explicita estructura, comportamiento, visibilidad y multiplicidades.

7.7 Decisiones técnicas justificadas

Decisión	Justificación
Atributos privados (-).	Favorecen el encapsulamiento.
Operaciones públicas (+).	Representan comportamientos invocables desde otras partes del sistema.
Integer para identificadores/cantidades.	Valores enteros de identificación y conteo.
String para texto, documento y teléfono.	Adecuado para texto; documento/teléfono preservan ceros iniciales y formatos.
Decimal para importes.	Representa valores monetarios con parte decimal.
Date para fechas.	Representa fechas de venta y comprobante.
Composición Venta-Detalle.	Los detalles forman parte de una venta; una venta tiene uno o más detalles.
Venta-Comprobante 0..1.	El comprobante puede ser opcional y único por venta.
No usar #.	No se definieron clases heredadas.

8. Semana 07: Herencia, composición e interfaces

Actividad: identificación de comportamientos compartidos, elección de alternativa de diseño, implementación en Python, pruebas y justificación técnica.

8.1 Análisis de las clases del proyecto

Para esta actividad se revisaron las seis clases del Sistema de Gestión para una Tienda de Ropa: Usuario, Producto, Cliente, Venta, Detalle de Venta y Comprobante. El análisis se realizó tomando como base el modelo de clases previamente validado.

Alternativa	Análisis en el proyecto	Decisión
Herencia	No se identifica una relación natural de tipo "es un" entre las clases. No corresponde crear una jerarquía únicamente para reutilizar atributos o métodos.	No aplicar
Composición	Venta contiene uno o más Detalles de Venta. El detalle forma parte de una venta concreta y participa en el cálculo de su total.	Aplicar
Interfaz	Las clases actuales no requieren compartir un contrato común que justifique una interfaz independiente de la jerarquía de clases.	No aplicar

8.2 Alternativa seleccionada: composición

Se seleccionó la **composición** para representar la relación entre las clases **Venta** y **Detalle de Venta**. Esta alternativa representa una relación de tipo "tiene un": una Venta tiene uno o más Detalles de Venta. Cada detalle registra producto, cantidad, precio y subtotal.

Relación	Tipo	Multiplicidad	Interpretación
Venta — Detalle de Venta	Composición	1 — 1..*	Una Venta contiene uno o más detalles.

Representación UML: **Venta** ◆■ **Detalle de Venta**

El rombo relleno se coloca en el lado de Venta porque Venta representa el "todo" y Detalle de Venta representa la "parte".

8.3 Justificación técnica

No se aplica herencia porque ninguna de las clases presenta una relación natural de tipo "es un". Heredar solamente para reutilizar código produciría una jerarquía artificial. Tampoco se utiliza una interfaz porque el modelo no presenta varias clases independientes que necesiten implementar un mismo contrato de comportamiento.

La composición resulta adecuada porque Venta administra sus objetos Detalle de Venta. Esta decisión mantiene la estructura del modelo del dominio, representa correctamente la relación todo-parte y evita agregar complejidad innecesaria.

Esta elección coincide con el criterio trabajado en clase: la herencia se utiliza cuando existe una relación "es un", mientras que la composición se utiliza cuando existe una relación "tiene un".

8.4 Implementación en Python

Para comprobar que la decisión de composición se refleja en el comportamiento del modelo, se implementó un ejemplo sencillo en Python. La clase Venta administra una colección de objetos DetalleVenta y calcula el total de la venta.

```
class DetalleVenta:
    def __init__(self, producto, cantidad, precio):
        self.producto = producto
        self.cantidad = cantidad
        self.precio = precio
    def subtotal(self):
        return self.cantidad * self.precio

class Venta:
    def __init__(self):
        self.detalles = []
    def agregar_detalle(self, producto, cantidad, precio):
        detalle = DetalleVenta(producto, cantidad, precio)
        self.detalles.append(detalle)
    def calcular_total(self):
        return sum(d.subtotal() for d in self.detalles)

venta = Venta()
venta.agregar_detalle("P001", 2, 50)
venta.agregar_detalle("P002", 1, 30)
print("Cantidad de detalles:", len(venta.detalles))
print("Total:", venta.calcular_total())
```

El código muestra que los objetos DetalleVenta son creados y administrados por una instancia de Venta. Esto permite comprobar en Python la relación de composición planteada en el diagrama UML.

8.5 Verificación mediante pruebas

Prueba realizada	Resultado esperado	Resultado obtenido
Registrar dos detalles en una Venta	2 detalles	2 detalles
Calcular el total	130.00	130.00
Verificar la relación	Venta administra sus detalles	Comprobado

La prueba confirma que una Venta puede contener varios Detalles de Venta y que cada detalle participa en el cálculo del total. El resultado obtenido fue **130.00**, correspondiente a 2 unidades de P001 a 50 y 1 unidad de P002 a 30.

8.6 Resultado del análisis

Elemento	Resultado
Comportamientos compartidos	Se revisaron las seis clases del modelo y no se identificó una jerarquía de herencia necesaria.
Relación todo-parte	Venta — Detalle de Venta.
Alternativa elegida	Composición.
Alternativas descartadas	Herencia e interfaz, por no corresponder al modelo actual.
Comprobación	Implementación en Python y prueba con dos detalles y total 130.00.

La actividad mantiene las clases y responsabilidades definidas previamente y añade el análisis de diseño, la implementación y la prueba solicitados para la Semana 07.

8.7 Bitácora de uso de inteligencia artificial

La inteligencia artificial generativa se utilizó como apoyo para analizar las alternativas de diseño y proponer una implementación sencilla en Python. Las propuestas fueron contrastadas con el modelo previamente validado, las indicaciones del docente y las necesidades del sistema. Las decisiones finales corresponden a la estudiante.

Uso / consulta	Resultado obtenido	Decisión final
Analizar las clases del proyecto.	Se revisaron Usuario, Producto, Cliente, Venta, Detalle de Venta y Comprobante.	Mantener las seis clases del modelo validado.
Comparar herencia, composición e interfaz.	Se identificó que Venta — Detalle de Venta representa una relación "tiene un".	Seleccionar composición.
Justificar la alternativa.	La composición representa una relación todo-parte y evita una jerarquía artificial.	Aplicar composición.
Proponer implementación en Python.	Se generó un ejemplo con Venta y DetalleVenta.	Revisar, adaptar y utilizar el código como prueba.
Proponer pruebas.	Se comprobó la cantidad de detalles y el cálculo del total.	Aceptar la prueba después de verificar los resultados.

8.8 Verificación de consistencia de la actividad

Elemento	Resultado
Relaciones simples	Usuario — Venta, Cliente — Venta, Producto — Detalle de Venta y Venta — Comprobante se mantienen como asociaciones simples.
Relación todo-parte	Venta — Detalle de Venta se identifica como relación todo-parte.
Tipo elegido	Composición, representada mediante rombo relleno del lado de Venta.
Agregación	No se incorpora porque ninguna relación actual cumple el criterio de una parte independiente del todo.
Multiplicidades	Se conservan las multiplicidades previamente validadas: 1, 0..*, 1..* y 0..1.
Implementación	La prueba registra dos detalles dentro de una Venta y obtiene un total de 130.00.
Diagrama vs. implementación	La relación de composición del modelo UML se refleja en la colección de detalles administrada por Venta.

8.9 Conclusión de la actividad

El análisis permitió determinar que la composición es la alternativa que corresponde al modelo actual del sistema. La implementación y las pruebas en Python verifican el comportamiento de Venta y DetalleVenta, mientras que la bitácora registra el apoyo de IA generativa y la revisión de las decisiones.

8.10 Evidencia de la actividad de Semana 07

La siguiente evidencia resume la decisión de diseño aplicada al proyecto:

Clase / relación	Decisión	Fundamento
Usuario — Venta	Asociación	Ambas clases representan objetos independientes; Usuario registra ventas.
Cliente — Venta	Asociación	Ambas clases pueden existir de forma independiente; la venta se relaciona con el cliente.
Producto — Detalle de Venta	Asociación	Producto existe independientemente y puede aparecer en diferentes detalles.
Venta — Detalle de Venta	Composición	Detalle de Venta forma parte de una Venta concreta; multiplicidad 1..*.
Venta — Comprobante	Asociación	El comprobante puede ser opcional; multiplicidad 0..1.

Criterio técnico aplicado

Criterio	Aplicación en el proyecto
"Es un"	No se identificó una relación que requiera herencia.
"Tiene un"	Venta tiene Detalle de Venta; se aplica composición.
Contrato común	No se identificó una necesidad de interfaz entre las clases actuales.
Evitar sobreingeniería	No se agregan jerarquías ni interfaces sin una necesidad real.

Correspondencia con la tarea de Semana 07

Requisito de la actividad	Evidencia en el informe
Identificar clases que comparten comportamiento.	Análisis de las seis clases del proyecto.
Decidir entre herencia, composición o interfaz.	Selección técnica de composición.
Implementar al menos un ejemplo en Python.	Clases Venta y DetalleVenta.
Verificar mediante pruebas.	Dos detalles registrados y total 130.00.
Justificar técnicamente.	Análisis de "es un", "tiene un" y evitación de sobreingeniería.
Registrar el análisis en bitácora de IA.	Tabla de consultas, resultados y decisiones finales.

Con esta sección, el informe integra la actividad de la Semana 07 sobre herencia, abstracción e interfaces con el modelo UML previamente desarrollado, manteniendo la continuidad del mismo proyecto.

Resultado: correspondencia entre problema, entidades, responsabilidades, requerimientos, historias y diagramas.

10. Conclusiones

1. Se identificaron seis entidades principales: Usuario, Producto, Cliente, Venta, Detalle de Venta y Comprobante.
2. Las entidades se describieron mediante atributos, responsabilidades y relaciones.
3. Los requerimientos funcionales y no funcionales mantienen prioridades y entidades relacionadas.
4. Las cuatro historias de usuario incluyen criterios de aceptación verificables.
5. Los diagramas UML presentan actores, funcionalidades, estructura y secuencia del registro de venta.
6. La actividad ampliada documenta tipos, visibilidades, operaciones y multiplicidades con justificación.
7. La bitácora de IA y la verificación de consistencia registran el apoyo y las revisiones realizadas.

11. Evidencias visuales y entrega

El informe contiene los diagramas y tablas de modelado. La imagen del diagrama de clases se ha revisado para mejorar la lectura de las asociaciones, multiplicidades y responsabilidades.

Evidencia	Contenido	Estado
01 Modelo del dominio	Seis entidades y relaciones.	Figura incluida.
02 Casos de uso	Actores y funcionalidades.	Figura incluida.
03 Diagrama de clases	Clases, atributos, operaciones y asociaciones.	Incluido y revisado.
04 Secuencia/historias	Flujo de venta o historias.	Secuencia e historias incluidas.
Capturas reales de herramienta UML	Ventana del software utilizado.	Agregar si es requisito del docente.

12. Referencias de trabajo

- Material de clase POO, Semana 04: modelado del dominio y especificación de requerimientos.
- Material de clase POO, Semana 05: UML aplicado a POO: clases, atributos, métodos y visibilidad.
- Guía práctica y rúbrica de las actividades de modelado y UML.
- Modelo del dominio y requerimientos validados del Sistema de Gestión para una Tienda de Ropa.